

Taller 1: Perceptrón y Adaline

Informe de Resultados

Asignatura: Inteligencia Computacional Aplicada
Docente: Cesar Andrey Perdomo Charry
Estudiante: Sergio Santiago Duarte Rojas
Código: 20261595002
Fecha: 17 de septiembre de 2026

Índice

1. Introducción	2
2. Marco teórico (resumen)	2
2.1. El Perceptrón Simple y su función de activación	2
2.2. Las tres reglas de ajuste de pesos	2
2.3. El Adaline, la Regla Delta y la función de costo continua	3
3. Metodología	4
3.1. Implementación	4
3.2. Parámetros utilizados	4
3.3. Datasets empleados	5
3.4. Protocolo experimental	5
4. Resultados	6
4.1. Perceptrón con las tres reglas de ajuste	6
4.2. Efecto de la tasa de aprendizaje y del umbral en el Perceptrón	6
4.3. Efecto de los pesos iniciales en el Perceptrón	7
4.4. Adaline sobre compuertas AND y OR	8
4.5. Efecto de los pesos iniciales en el Adaline	9
4.6. Dinámica del error de los dos modelos	9
4.7. Control negativo con la compuerta XOR	10
4.8. Aplicación a los conjuntos que genera <code>Dataset_train_test</code>	11
4.9. Aplicación al conjunto de billetes	12
5. Análisis y discusión	13
5.1. Convergencia de las tres reglas de aprendizaje	13
5.2. Comportamiento ante cambios del umbral y de la tasa de aprendizaje	14
5.3. Diferencias en la forma de la curva de error	14
5.4. Efecto del punto de partida de los pesos	15
5.5. Comportamiento sobre datos reales	15
6. Conclusiones	16
7. Referencias	16
A. Tablas complementarias	17
B. Código de los bloques completados	17

1. Introducción

Este informe documenta la implementación y la evaluación de los dos modelos de neurona artificial más antiguos que siguen presentes en la enseñanza de redes neuronales, el Perceptrón Simple propuesto por Rosenblatt en 1958 y el Adaline propuesto por Widrow y Hoff en 1960. Ambos resuelven el mismo tipo de problema, la clasificación binaria de patrones separables por un hiperplano, y comparten la misma arquitectura de una sola neurona con una suma ponderada de las entradas más un sesgo. La diferencia está en la señal que cada uno usa para corregir los pesos. El Perceptrón corrige a partir del error de la salida binaria, después de aplicar la función escalón, mientras que el Adaline corrige a partir del error de la salida lineal, antes del escalón. Esa diferencia, que en las fórmulas ocupa un solo símbolo, produce dos comportamientos de entrenamiento distintos y explica casi todos los resultados que se reportan más adelante.

El trabajo se desarrolló sobre el *live script* base entregado por el docente, en el cual se completaron los cuatro bloques marcados como pendientes sin alterar los nombres ni los argumentos de las funciones, tal como pide el enunciado. Sobre esa base se agregaron los experimentos necesarios para responder los doce numerales del taller. Los modelos se prueban primero sobre compuertas lógicas AND y OR de 2, 3 y 4 entradas, que son linealmente separables y por lo tanto permiten verificar que la implementación es correcta, y después sobre el conjunto de autenticación de billetes con las cuatro proporciones de partición solicitadas. El informe cierra con una discusión sobre el efecto de cada parámetro del modelo, incluido el punto de partida de los pesos, que resultó ser el factor que más diferencia a los dos modelos entre sí.

2. Marco teórico (resumen)

2.1 El Perceptrón Simple y su función de activación

La neurona recibe n entradas y una entrada constante igual a uno cuyo peso actúa como sesgo. La suma ponderada de la ecuación (1) se conoce como entrada neta.

$$net = \sum_{i=1}^n w_i x_i + b \quad (1)$$

La salida se obtiene aplicando la función escalón de la ecuación (2), que compara la entrada neta contra un umbral θ y entrega una decisión binaria.

$$Y(x) = \begin{cases} 1 & \text{si } net \geq \theta \\ 0 & \text{si } net < \theta \end{cases} \quad (2)$$

Geométricamente la condición $net = \theta$ define un hiperplano en el espacio de entrada, de modo que la neurona solo puede separar dos clases cuando existe un hiperplano que las deja a lados opuestos. El umbral y el sesgo cumplen papeles equivalentes, porque desplazar θ produce el mismo efecto que restar esa cantidad al sesgo, y esa redundancia se aprecia en los resultados experimentales.

2.2 Las tres reglas de ajuste de pesos

El taller pide dejar el modelo parametrizable para elegir entre tres formas de corregir los pesos, numeradas 1, 2 y 3 en el enunciado y recogidas en las ecuaciones (3) a (5). En los tres casos el ajuste

final es $w_i^* = w_i + \Delta w_i$.

$$\Delta w_i = d(x) x_i \quad (3)$$

$$\Delta w_i = [d(x) - Y(x)] x_i \quad (4)$$

$$\Delta w_i = \alpha [d(x) - Y(x)] x_i \quad (5)$$

La diferencia conceptual entre ellas es más grande de lo que sugiere su parecido. La plantilla del curso llama a la regla 1 regla tipo hebbiana, nombre que viene del psicólogo Donald Hebb, quien propuso en 1949 que la conexión entre dos neuronas se refuerza cada vez que ambas se activan al mismo tiempo. Trasladada a la fórmula, la idea es que el peso w_i crece cuando la entrada x_i y la salida deseada $d(x)$ valen uno a la vez, sin importar lo que haya respondido la red. La regla no consulta entonces la salida actual, aplica el mismo incremento cada vez que ve un patrón, acierte o no, y por eso carece de un mecanismo que la detenga. Es una regla de asociación, no de corrección de error. La regla 2 es la formulación original del Perceptrón y sí usa el error, por lo cual el incremento se anula exactamente cuando la clasificación es correcta. La regla 3 es la misma con una tasa de aprendizaje explícita que gradúa el tamaño del paso. Para las dos últimas, el teorema de convergencia del Perceptrón garantiza que si las clases son linealmente separables el algoritmo encuentra un hiperplano separador tras un número finito de correcciones, sin importar el punto de partida.

2.3 El Adaline, la Regla Delta y la función de costo continua

El Adaline conserva la ecuación (1) pero cambia dos cosas. Durante el entrenamiento la salida que se compara contra el valor deseado es la salida lineal $Y(x) = net$, sin escalón, y la función escalón se aplica únicamente al final para convertir esa salida continua en una decisión de clase. El entrenamiento pasa entonces a ser la minimización de la función de costo de la ecuación (6).

$$E = \frac{1}{2} \sum_x (d(x) - Y(x))^2 \quad (6)$$

Derivando esa función respecto de cada peso y avanzando en sentido contrario al gradiente se obtiene la Regla Delta, también conocida como regla LMS, que aparece en la ecuación (7).

$$\Delta w_i = \alpha [d(x) - Y(x)] x_i \quad (7)$$

La expresión (7) es idéntica a la de la regla 3, y sin embargo los dos algoritmos se comportan de forma distinta porque $Y(x)$ significa cosas diferentes en cada una. En el Perceptrón $Y(x)$ es la salida del escalón, una función que solo toma dos valores y cuya derivada es cero en casi todo el dominio e indefinida en el umbral. No existe entonces una superficie de error sobre la cual descender, y lo único que el algoritmo puede hacer es contar patrones mal clasificados, un número entero que cambia a saltos. En el Adaline $Y(x)$ es la salida lineal, de modo que E es una función cuadrática de los pesos, continua y diferenciable, cuya gráfica tiene forma de cuenco con un único punto más bajo. Descender por esa superficie lleva siempre al mismo mínimo.

De esa distinción se derivan tres consecuencias que aparecen una y otra vez en los resultados. El Perceptrón se detiene en cuanto clasifica bien y el Adaline sigue ajustando aunque la clasificación ya sea correcta, el mínimo de E no coincide necesariamente con el hiperplano que separa las clases, y el descenso de gradiente tiene un límite de estabilidad para la tasa de aprendizaje que el Perceptrón no puede presentar.

3. Metodología

3.1 Implementación

Todo el trabajo se desarrolló sobre el *live script* base del docente, que define ocho funciones locales y deja cuatro con bloques pendientes. Se completó `init_perceptron` con la inicialización aleatoria de pesos y sesgo, `compute_delta` con las tres reglas de las ecuaciones (3) a (5), `train_adaline` con la Regla Delta de la ecuación (7) y `split_dataset` con la partición aleatoria en entrenamiento y prueba. El código de los cuatro bloques se reproduce en el Anexo B. No se modificó ningún nombre ni ningún argumento, de modo que el script conserva la estructura original.

Conviene dejar constancia de un detalle de la plantilla que afecta la interpretación de los resultados. La función `train_perceptron` ajusta el sesgo con $b \leftarrow b + [d(x) - Y(x)]$ cualquiera que sea la regla seleccionada, es decir con paso unitario y sin el factor α . El sesgo no se comporta entonces como un peso más de una entrada constante, que es lo que sugiere la figura del enunciado, y en la regla 1 tampoco es hebbiano. Como el enunciado pide no modificar las funciones de la plantilla, esa línea se conservó intacta y los resultados de este informe corresponden a esa formulación.

Para poder ejecutar los reportes de aprendizaje que piden los numerales 3 y 9 se agregaron tres campos a las estructuras de configuración, la escala de los pesos iniciales, el número de semillas por experimento y el interruptor de escalado de las entradas, y dos funciones locales nuevas, `accuracy_of` para calcular la exactitud de un par de pesos sobre un conjunto y `scale_minmax` para reescalar las entradas con las cotas del conjunto de entrenamiento. Las secciones de experimentos se numeraron como subsecciones de las que trae la plantilla, de manera que la sección 7.2 desarrolla el barrido de la tasa de aprendizaje que la sección 7 menciona, y así sucesivamente.

3.2 Parámetros utilizados

La Tabla 1 recoge la configuración base de los dos modelos. Cada experimento modifica un solo campo y deja los demás en el valor de esta tabla, de modo que el efecto observado se pueda atribuir al parámetro variado.

Tabla 1: Parámetros del Perceptrón y Adaline

Parámetro	Perceptrón	Adaline
Umbral θ	0.5	0.5
Regla de aprendizaje	las tres del enunciado	Regla Delta
Tasa de aprendizaje α	0.1	0.05
Épocas máximas	100	200
MSE objetivo	no aplica	0.0010
Escala de los pesos iniciales	0.5	0.5
Semillas por experimento	10	10
Escalado en datos reales	min-max	min-max

Los valores explorados en los barridos fueron $\alpha \in \{0.01, 0.05, 0.1, 0.5, 1\}$ para el Perceptrón en compuertas y $\{0.001, 0.01, 0.05, 0.1, 0.5, 1\}$ para el Adaline, umbrales $\{0, 0.5, 1\}$ y $\{0.3, 0.5, 0.7\}$ respectivamente, y escalas de pesos iniciales $\{0, 0.001, 0.5, 5\}$ en los dos modelos. Sobre el conjunto real se usaron $\alpha \in \{0.01, 0.1, 1\}$ para el Perceptrón y $\{0.0001, 0.001, 0.01, 0.05\}$ para el Adaline, con valores más pequeños en este último por la magnitud de las entradas sin escalar.

3.3 Datasets empleados

Se emplearon tres conjuntos de datos. El primero lo forman las compuertas lógicas AND y OR de 2, 3 y 4 entradas, generadas como tabla de verdad completa con 2^n patrones, es decir 4, 8 y 16 filas. La compuerta AND produce salida uno solamente cuando todas las entradas valen uno, de manera que la clase positiva tiene un único representante sin importar el número de entradas. La compuerta OR produce salida uno cuando al menos una entrada vale uno, y por lo tanto la clase negativa es la que tiene un único representante. Ese desbalance espejo entre las dos compuertas resulta determinante para explicar el comportamiento de la primera regla de aprendizaje. Se incluyó además la compuerta XOR de 2 entradas como control negativo, por no ser linealmente separable.

El segundo conjunto es Iris, el que carga el script de particiones entregado en clase, con 150 flores, cuatro medidas en centímetros y tres especies de cincuenta ejemplares cada una. Sobre él se plantea el problema binario de *setosa* contra el resto, que es linealmente separable, y que da respuesta a los numerales 4 y 10 del enunciado.

El tercer conjunto es `data_banknote_authentication.txt`, que contiene 1372 billetes descritos por cuatro estadísticos obtenidos de la transformada wavelet de su imagen. La Tabla 2 resume esos atributos. Las cuatro variables se mueven en escalas comparables entre sí en cuanto al orden de magnitud, pero la curtosis alcanza un máximo de 17.93 frente a un máximo de 2.45 de la entropía, diferencia suficiente para afectar al Adaline, cuya corrección es proporcional al valor de la entrada. La clase está razonablemente balanceada, con 762 billetes auténticos y 610 falsos.

Tabla 2: Estadísticos descriptivos de los atributos del conjunto de billetes

Atributo	Mínimo	Máximo	Media	Desviación
Varianza	-7.04	6.82	0.43	2.84
Asimetría	-13.77	12.95	1.92	5.87
Curtosis	-5.29	17.93	1.40	4.31
Entropía	-8.55	2.45	-1.19	2.10

Las particiones se generan con la función `split_dataset`, que mezcla los índices con una semilla fija y corta en la proporción pedida, con lo cual se obtienen conjuntos de entrenamiento de 823, 960, 1098 y 1235 patrones y conjuntos de prueba de 549, 412, 274 y 137. El punto de corte se redondea porque el producto de la proporción por 1372 no siempre es entero.

3.4 Protocolo experimental

En las compuertas el resultado depende de los pesos iniciales, de modo que cada configuración se repite con diez semillas y se reporta el promedio de épocas, el porcentaje de ejecuciones que convergieron dentro del tope y la exactitud promedio de los pesos finales. Se considera que el Perceptrón converge cuando completa una época entera sin necesitar ninguna corrección. En el conjunto de billetes se usa una sola semilla por configuración, porque el número de patrones hace que la variación entre semillas sea mucho menor que entre particiones, y se reporta la exactitud sobre entrenamiento y sobre prueba. El número de épocas se omite de esas tablas porque es constante, el Perceptrón agota siempre su tope de cien y el Adaline el suyo de doscientas. Las entradas se escalan al rango cero a uno con los mínimos y máximos calculados únicamente sobre el conjunto de entrenamiento, y esas mismas cotas se aplican al conjunto de prueba para no filtrar información.

4. Resultados

4.1 Perceptrón con las tres reglas de ajuste

La Tabla 3 y la Figura 1 comparan las tres reglas sobre las seis compuertas con los parámetros base. Las reglas 2 y 3 convergen en la totalidad de las semillas y en todas las compuertas, con exactitud del cien por ciento. La regla hebbiana se comporta de forma radicalmente distinta según la compuerta. En OR converge en poco más de tres épocas, más rápido que las otras dos, mientras que en AND no converge en ninguna de las treinta ejecuciones y deja una exactitud entre 62.5 y 75.0 por ciento.

Tabla 3: Perceptrón sobre compuertas lógicas con los parámetros base. Promedio de diez semillas

Compuerta	Épocas promedio			Exactitud (%)		
	Regla 1	Regla 2	Regla 3	Regla 1	Regla 2	Regla 3
AND 2	100.0	5.9	13.7	67.5	100.0	100.0
AND 3	100.0	12.1	17.7	62.5	100.0	100.0
AND 4	100.0	12.3	15.6	75.0	100.0	100.0
OR 2	3.2	4.0	13.0	100.0	100.0	100.0
OR 3	3.3	4.3	16.8	100.0	100.0	100.0
OR 4	3.6	6.3	30.4	100.0	100.0	100.0

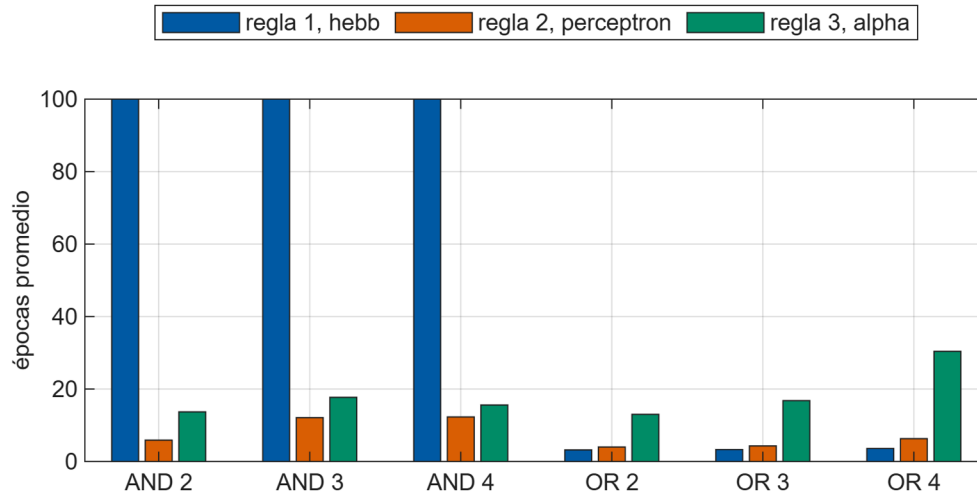


Figura 1: Épocas promedio hasta la convergencia con las tres reglas de ajuste. Las barras de la regla hebbiana en AND alcanzan el tope de cien épocas porque nunca convergen

Los pesos finales explican ese contraste mejor que las épocas. En la Tabla 4 las reglas 2 y 3 terminan con pesos del orden de la unidad, mientras que la regla hebbiana sobre la compuerta AND llega a valores cercanos a cien en los pesos y a menos cien en el sesgo, dos órdenes de magnitud por encima de la solución que encuentran las otras dos reglas para el mismo problema.

4.2 Efecto de la tasa de aprendizaje y del umbral en el Perceptrón

La Tabla 5 muestra que con la regla 3 el número de épocas disminuye de forma monótona al aumentar α en las seis compuertas. Con α igual a 0.01 se requieren entre 71.3 y 91.4 épocas, y con α igual a uno se requieren entre 4.0 y 12.3, valores que coinciden con los de la regla 2 en la Tabla 3 porque

Tabla 4: Pesos finales del Perceptrón en compuertas de dos entradas con la semilla uno

Regla	Compuerta	w_1	w_2	b	Épocas	Exactitud (%)
Regla 1	AND	99.675	100.590	-99.379	100	75.0
	OR	7.676	8.591	-0.379	4	100.0
Regla 2	AND	1.676	0.591	-1.379	4	100.0
	OR	1.676	1.591	-0.379	5	100.0
Regla 3	AND	1.176	0.791	-1.379	17	100.0
	OR	0.975	0.891	-0.379	18	100.0

ambas expresiones se vuelven la misma cuando la tasa vale uno. Ninguna de las configuraciones presentó inestabilidad, lo cual confirma que en el Perceptrón una tasa alta no degrada la solución.

Tabla 5: Perceptrón con la regla 3. Épocas promedio de diez semillas según la tasa de aprendizaje

Compuerta	Tasa de aprendizaje α				
	0.01	0.05	0.1	0.5	1
AND 2	71.3	24.2	13.7	7.4	5.9
AND 3	79.2	29.0	17.7	10.7	12.1
AND 4	80.1	24.7	15.6	14.8	12.3
OR 2	77.4	23.5	13.0	4.7	4.0
OR 3	75.2	30.9	16.8	5.7	4.3
OR 4	91.4	57.0	30.4	8.8	6.3

El umbral, en cambio, apenas altera el resultado. La Tabla 16 del anexo recoge el barrido completo. Ninguna de las seis compuertas muestra una tendencia consistente al mover el umbral, y la mayor diferencia aparece en OR de 4 entradas, que pasa de 15.8 épocas con θ igual a cero a 30.4 con 0.5 y vuelve a 15.0 con uno. El umbral no cambia el hiperplano al que se llega, solo el número de correcciones necesarias para alcanzarlo, porque el sesgo absorbe durante las primeras épocas cualquier desplazamiento del umbral.

4.3 Efecto de los pesos iniciales en el Perceptrón

La Tabla 6 y la Figura 2 presentan el experimento sobre el punto de partida de los pesos. Con pesos iniciales exactamente cero, la compuerta AND converge en cinco épocas y la OR en doce, con la totalidad de las semillas y con pesos finales idénticos entre ellas, lo cual es esperable porque con escala cero todas las semillas parten del mismo punto. Lo relevante es que con escala 0.001 el resultado se mantiene igual, con un rango de dispersión de los pesos finales de 0.004 en AND, lo cual indica que la trayectoria queda determinada por los datos y no por el azar de la inicialización.

A medida que la escala crece el comportamiento se degrada de forma progresiva. Con escala 0.5, que es el valor de la plantilla, se necesitan trece épocas y los pesos finales varían alrededor de una unidad entre semillas. Con escala cinco se superan las cuarenta épocas, una de cada diez semillas agota el tope sin converger y el rango de los pesos finales supera las diez unidades, es decir que cada semilla termina en un hiperplano distinto.

Tabla 6: Efecto de la escala de los pesos iniciales en el Perceptrón con la regla 3. Promedio de diez semillas. El rango es la diferencia entre el mayor y el menor valor final de w_1 entre semillas

Compuerta	Escala inicial	Épocas	Convergió (%)	Exactitud (%)	Rango de w_1
AND 2	0	5.0	100.0	100.0	0.000
	0.001	5.0	100.0	100.0	0.004
	0.5	13.7	100.0	100.0	1.041
	5	44.2	90.0	95.0	10.314
OR 2	0	12.0	100.0	100.0	0.000
	0.001	12.7	100.0	100.0	0.100
	0.5	13.0	100.0	100.0	1.048
	5	41.3	90.0	97.5	11.375

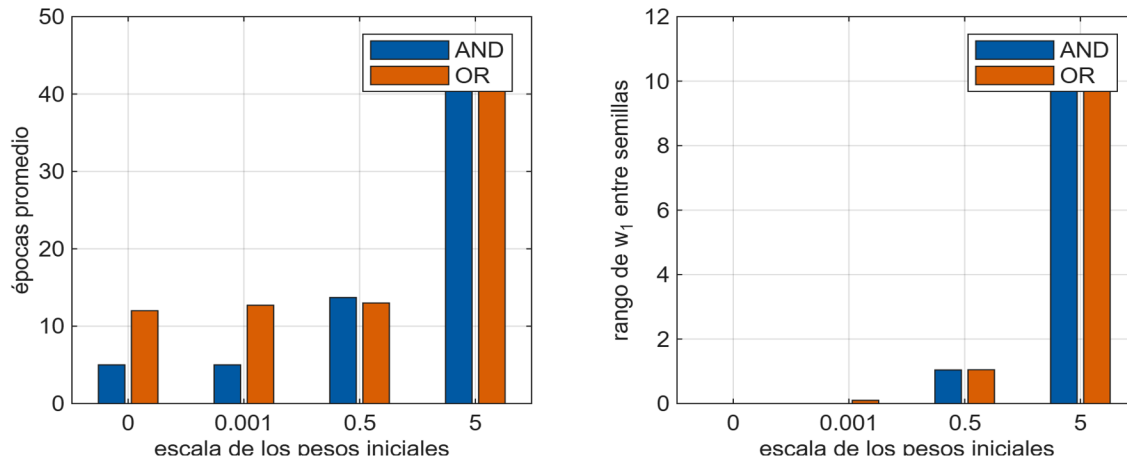


Figura 2: Efecto de la escala de los pesos iniciales en el Perceptrón con la regla 3. A la izquierda las épocas promedio hasta converger y a la derecha la dispersión de los pesos finales entre semillas

4.4 Adaline sobre compuertas AND y OR

La Tabla 7 resume el Adaline sobre las seis compuertas con la tasa base y la Figura 3 muestra las curvas de error correspondientes. El MSE desciende de forma suave y monótona y se estabiliza antes de la época sesenta en un valor entre 0.0231 y 0.0369. Como la salida lineal de una única neurona no puede reproducir de manera exacta los ceros y unos de una tabla de verdad, ese piso positivo es el mínimo alcanzable y el criterio de parada por MSE objetivo nunca se activa, de modo que el entrenamiento agota las doscientas épocas en todos los casos. La clasificación es perfecta en AND de 2 y 3 entradas y en OR de 2, mientras que AND de 4 y OR de 3 y 4 dejan un patrón mal clasificado.

La tasa de aprendizaje determina dónde se detiene el descenso, como recoge la Tabla 8. Con 0.001 el paso es tan pequeño que doscientas épocas no bastan para llegar al mínimo del error, el MSE se queda entre 0.0430 y 0.1082 y la exactitud cae hasta el cincuenta por ciento en AND de 2 entradas. En el intervalo de 0.01 a 0.1 el MSE alcanza su mínimo y la exactitud es la mejor de la tabla. Con 0.5 el error final ya es entre dos y cinco veces mayor que el mínimo, señal de que el descenso empieza a oscilar alrededor del óptimo, y con uno el proceso diverge, es decir que el error crece de una época a la siguiente sin detenerse, hasta que los pesos dejan de representar un modelo útil. Por eso la tabla marca esa columna como divergente en lugar de reportar un valor.

El umbral de decisión, que en el Adaline no interviene en el ajuste de los pesos sino solo en la conversión final de la salida lineal en clase, permite recuperar algunos de los patrones perdidos sin

Tabla 7: Adaline sobre compuertas lógicas con la tasa base de 0.05 y doscientas épocas. El umbral se aplica sobre un mismo modelo ya entrenado, así que las tres columnas de exactitud provienen de un único entrenamiento

Compuerta	MSE final	Exactitud (%) según el umbral θ		
		0.3	0.5	0.7
AND 2	0.0346	100.0	100.0	100.0
AND 3	0.0369	100.0	100.0	87.5
AND 4	0.0268	100.0	93.8	93.8
OR 2	0.0346	100.0	100.0	100.0
OR 3	0.0341	87.5	87.5	100.0
OR 4	0.0231	93.8	93.8	93.8

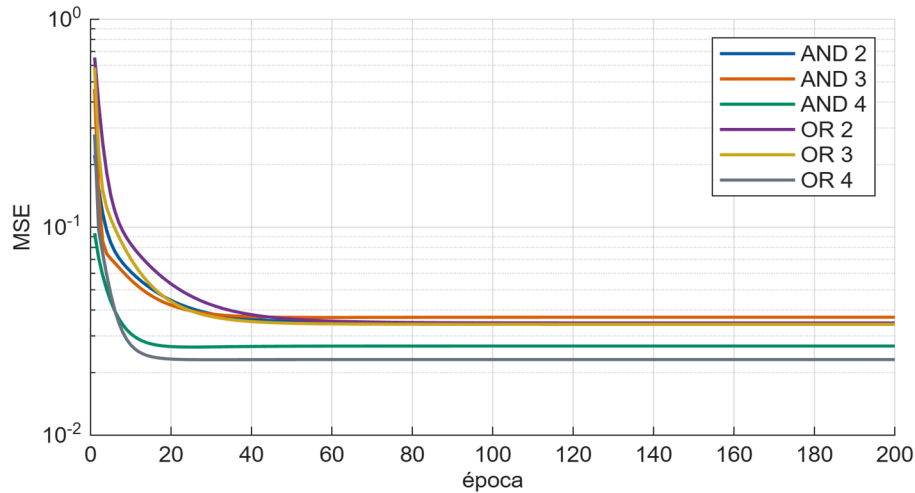


Figura 3: Curva de error de entrenamiento del Adaline sobre las seis compuertas con la tasa de aprendizaje base. La escala vertical es logarítmica

volver a entrenar, como muestran las tres últimas columnas de la Tabla 7. La compuerta AND de 4 entradas alcanza el cien por ciento con umbral 0.3 y la OR de 3 entradas lo alcanza con umbral 0.7.

4.5 Efecto de los pesos iniciales en el Adaline

El mismo experimento de la Tabla 6 aplicado al Adaline produce el resultado opuesto, que recoge la Tabla 9. Partiendo de escalas que van de cero a cinco, el modelo termina siempre en los mismos pesos, con diferencias de una milésima, y con el mismo MSE final. La dispersión entre semillas crece con la escala inicial, pero lo hace de forma marginal, desde cero hasta 0.0102 en el caso más extremo, tres órdenes de magnitud por debajo de la dispersión que presentaba el Perceptrón en las mismas condiciones.

4.6 Dinámica del error de los dos modelos

La Figura 4 presenta las dos formas de error sobre la compuerta AND de dos entradas en paneles separados, porque un conteo entero de patrones y un error cuadrático medio no comparten unidad ni escala y no deben leerse sobre el mismo eje. Lo que se compara entre ambos paneles es la forma de la trayectoria, no su magnitud. La curva del Adaline es continua y decreciente en todo momento,

Tabla 8: Adaline según la tasa de aprendizaje, tras doscientas épocas. La marca diverge indica que el error creció sin control y los pesos dejaron de representar un modelo entrenado

Compuerta	Tasa de aprendizaje α					
	0.001	0.01	0.05	0.1	0.5	1
<i>MSE final</i>						
AND 2	0.0842	0.0334	0.0346	0.0386	0.1250	diverge
AND 3	0.0664	0.0323	0.0369	0.0439	0.1556	diverge
AND 4	0.0430	0.0225	0.0268	0.0332	0.1129	diverge
OR 2	0.1082	0.0329	0.0346	0.0386	0.1250	diverge
OR 3	0.0745	0.0319	0.0341	0.0372	0.0764	diverge
OR 4	0.0471	0.0218	0.0231	0.0246	0.0409	diverge
<i>Exactitud (%)</i>						
AND 2	50.0	100.0	100.0	100.0	75.0	diverge
AND 3	87.5	100.0	100.0	100.0	62.5	diverge
AND 4	93.8	93.8	93.8	100.0	56.3	diverge
OR 2	75.0	100.0	100.0	100.0	75.0	diverge
OR 3	75.0	100.0	87.5	87.5	87.5	diverge
OR 4	100.0	93.8	93.8	93.8	93.8	diverge

Tabla 9: Efecto de la escala de los pesos iniciales en el Adaline sobre compuertas de dos entradas. Promedio de diez semillas. El rango se define igual que en la Tabla 6

Compuerta	Escala inicial	w_1	w_2	b	MSE final	Rango de w_1
AND 2	0	0.526	0.513	-0.262	0.0346	0.0000
	0.001	0.526	0.513	-0.262	0.0346	0.0000
	0.5	0.526	0.513	-0.262	0.0346	0.0010
	5	0.526	0.513	-0.263	0.0346	0.0102
OR 2	0	0.473	0.487	0.264	0.0346	0.0000
	0.001	0.473	0.487	0.264	0.0346	0.0000
	0.5	0.473	0.487	0.263	0.0346	0.0010
	5	0.474	0.487	0.263	0.0346	0.0102

tal como corresponde a un descenso de gradiente sobre una función con un único mínimo. El conteo de errores del Perceptrón, en cambio, avanza a saltos y no es monótono, pues sube de dos a tres patrones mal clasificados en la tercera época, baja a uno, vuelve a subir a dos en la época trece y solo alcanza cero en la época dieciséis. La función declara la convergencia en la época diecisiete, una más, porque necesita recorrer una época completa sin aplicar ninguna corrección para poder afirmar que el hiperplano ya separa los datos.

4.7 Control negativo con la compuerta XOR

La Tabla 10 confirma que ninguno de los dos modelos resuelve la compuerta XOR. El Perceptrón agota las cien épocas sin converger y el Adaline se estabiliza en un MSE de 0.1385, cuatro veces el que alcanza sobre la AND, y los dos clasifican correctamente uno de los cuatro patrones. El resultado es el previsto por la teoría y sirve para descartar que los aciertos del resto del informe provengan de un error de implementación.

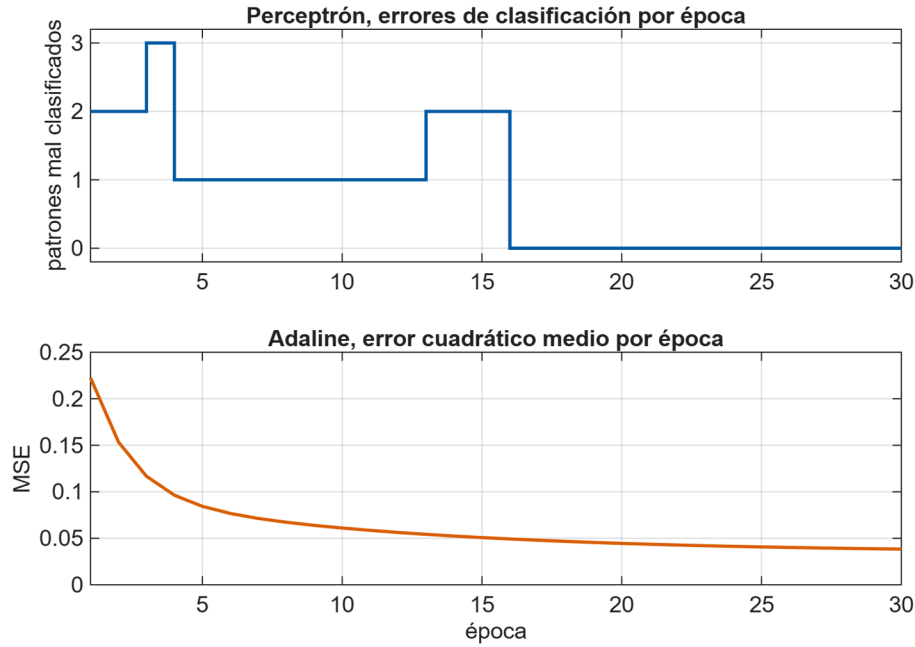


Figura 4: Dinámica del error sobre la compuerta AND de dos entradas. Arriba, patrones mal clasificados por el Perceptrón en cada época. Abajo, error cuadrático medio del Adaline en cada época

Tabla 10: Control negativo sobre la compuerta XOR de dos entradas

Modelo	Épocas	MSE final	Exactitud (%)
Perceptrón	100	no aplica	25.0
Adaline	200	0.1385	25.0

4.8 Aplicación a los conjuntos que genera Dataset_train_test

Los numerales 4 y 10 piden aplicar los modelos a los conjuntos que produce el script de particiones entregado en clase, que opera sobre Iris. Como la red tiene una sola neurona, el problema se plantea de forma binaria separando la especie *setosa* de las otras dos, que es el caso linealmente separable de ese conjunto. La Tabla 11 recoge el resultado de los dos modelos sobre las cuatro proporciones.

Tabla 11: Iris, *setosa* contra el resto, con las entradas escaladas. Perceptrón con la regla 3 y α igual a 0.1, Adaline con α igual a 0.01

Partición	Perceptrón		Adaline	
	Épocas	Prueba (%)	MSE final	Prueba (%)
60-40	2	100.0	0.0097	100.0
70-30	2	100.0	0.0106	100.0
80-20	2	100.0	0.0119	100.0
90-10	2	100.0	0.0116	100.0

El Perceptrón converge en dos épocas y clasifica sin error, tanto en entrenamiento como en prueba, en las cuatro particiones y con los tres valores de tasa probados. Es la confirmación más limpia del teorema de convergencia sobre un problema real, ya que las dos épocas corresponden a una pasada de correcciones y una segunda de verificación. El Adaline también alcanza el cien por

ciento con tasas de 0.01 y 0.05. Con tasas de 0.001 o menores baja a 98.3 y 97.8 por ciento en las particiones 60-40 y 70-30, que corresponden a un solo patrón mal clasificado en cada caso, porque doscientas épocas no bastan para llegar al mínimo del error con un paso tan pequeño. El contraste con el conjunto de billetes de la subsección siguiente es instructivo, allí ningún modelo converge porque las clases se traslapan, mientras que aquí el problema se agota en dos épocas.

4.9 Aplicación al conjunto de billetes

La Tabla 12 y la Figura 5 recogen el desempeño de los dos modelos sobre el conjunto de billetes con las entradas escaladas. El Perceptrón agota el tope de cien épocas en las cuatro particiones, lo cual indica que las dos clases no son perfectamente separables por un hiperplano, pero aun así alcanza entre 94.4 y 98.9 por ciento de exactitud sobre el conjunto de prueba. El Adaline se mueve en un intervalo algo más bajo y más estrecho, entre 94.9 y 97.3 por ciento. La distancia entre entrenamiento y prueba no supera los 2.5 puntos porcentuales en ninguna de las ocho combinaciones.

Tabla 12: Exactitud según la proporción de partición con las entradas escaladas. Perceptrón con la regla 3 y α igual a 0.1, Adaline con α igual a 0.01

Partición	Perceptrón (%)		Adaline (%)	
	Entrenamiento	Prueba	Entrenamiento	Prueba
60-40	95.0	94.4	97.9	97.3
70-30	99.3	98.5	97.5	95.9
80-20	98.8	98.9	97.9	96.7
90-10	99.1	97.8	97.2	94.9

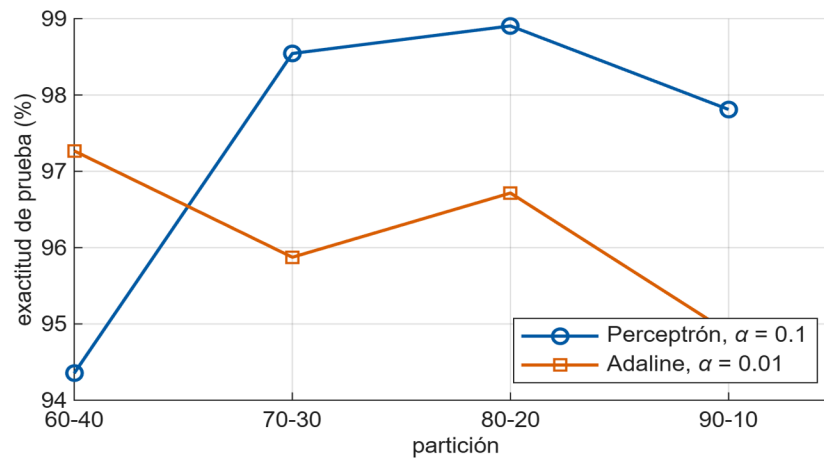


Figura 5: Exactitud sobre el conjunto de prueba de los dos modelos según la proporción de partición

La exactitud global oculta de qué lado se equivoca cada modelo. La Tabla 13 descompone los aciertos y errores sobre la partición 70-30, que tiene 412 billetes de prueba. El Perceptrón reparte sus seis errores entre las dos clases, con cuatro billetes auténticos marcados como falsos y dos falsos marcados como auténticos. El Adaline comete diecisiete errores, todos del mismo tipo, y no deja pasar ningún billete falso. En un escenario de detección de fraude esa asimetría importa más que el punto y medio de exactitud que los separa, porque el costo de aceptar un billete falso no es igual al de rechazar uno auténtico.

Tabla 13: Matriz de confusión sobre los 412 billetes de prueba de la partición 70-30. La clase positiva es el billete falso. VN son auténticos reconocidos, FP auténticos marcados como falsos, FN falsos que pasaron como auténticos y VP falsos detectados. La sensibilidad es la fracción de falsos detectados y la especificidad la fracción de auténticos reconocidos

Modelo	VN	FP	FN	VP	Sensibilidad (%)	Especificidad (%)
Perceptrón	236	4	2	170	98.8	98.3
Adaline	223	17	0	172	100.0	92.9

Los pesos con que terminan los dos modelos sobre esa misma partición, recogidos en la Tabla 14, muestran que ambos encuentran esencialmente la misma dirección de separación. Los tres primeros coeficientes del Perceptrón son entre 3.3 y 3.6 veces los del Adaline, y el sesgo guarda una proporción parecida de 3.1, de modo que los pesos de un modelo son aproximadamente un múltiplo de los del otro. Multiplicar todos los pesos y el sesgo por una misma cantidad positiva no cambia el signo de la suma ponderada, así que las dos fronteras de decisión son casi la misma. Los dos modelos coinciden además en asignar a la entropía un peso entre sesenta y ciento treinta veces menor que a las demás variables, lo cual sugiere que ese atributo aporta poco a la discriminación entre billetes auténticos y falsos.

Tabla 14: Pesos finales sobre la partición 70-30 con las entradas escaladas

Entrada	Perceptrón	Adaline
Varianza	-7.124	-1.960
Asimetría	-7.541	-2.090
Curtosis	-7.825	-2.375
Entropía	-0.110	-0.018
Sesgo	10.577	3.461

Los dos modelos se entrenaron además con las entradas sin escalar, y el resultado separa con claridad sus comportamientos. El Perceptrón no solo tolera la falta de escalado sino que mejora ligeramente, con exactitudes de prueba entre 97.1 y 99.3 por ciento frente a 94.4 y 98.9 con las entradas escaladas, y su mejor resultado global, 99.3 por ciento en la partición 90-10, se obtiene precisamente sin escalar. El Adaline hace lo contrario. Sin escalar y con la tasa base diverge en las cuatro particiones, y la red termina asignando todos los billetes a la misma clase, de modo que su exactitud, entre 56.6 y 60.6 por ciento, es solo la proporción de billetes auténticos en cada conjunto de prueba. Con una tasa de 0.01 el error ya no diverge pero el resultado es errático, entre 72.3 y 96.4 por ciento, y solo con tasas iguales o menores a 0.001 se recuperan valores comparables a los de la Tabla 12. Con las entradas escaladas, en cambio, cualquier tasa entre 0.001 y 0.05 produce resultados equivalentes. Las Tablas 15 y 17 del anexo recogen el detalle de los dos modelos.

5. Análisis y discusión

5.1 Convergencia de las tres reglas de aprendizaje

Dos de las tres reglas convergieron en todos los casos y la más rápida fue la regla 2, con un intervalo de 4.0 a 12.3 épocas frente a 13.0 a 30.4 de la regla 3. La razón es directa. La regla 2 equivale a la regla 3 con la tasa fijada en uno, de modo que cada corrección tiene el tamaño de la entrada, mientras que con una tasa de 0.1 la corrección es diez veces menor y se necesitan diez veces más

correcciones para recorrer la misma distancia desde unos pesos iniciales de escala 0.5. Esa relación se aprecia en la Tabla 5, donde la columna de tasa uno reproduce los valores de la regla 2.

El caso interesante es el de la regla hebbiana, que no converge en AND, sí converge en OR y además en OR es la más rápida de las tres. La explicación está en la ausencia del término de error combinada con la estructura de cada compuerta. Como el incremento vale $d(x)x_i$, la regla suma un valor positivo cada vez que encuentra un patrón de clase positiva, aunque la neurona ya lo clasifique correctamente, y no hace nada con los patrones de clase negativa. La compuerta AND tiene un solo patrón positivo, el de todas las entradas en uno, de manera que los pesos crecen sin límite en esa dirección y el sesgo, que se corrige con el término de error y con paso unitario, no alcanza a compensar ese crecimiento, hasta llegar a los valores cercanos a cien de la Tabla 4 y a una neurona que se activa ante casi cualquier entrada. En la compuerta OR ocurre lo contrario, tres de los cuatro patrones son positivos y el único negativo se resuelve con el sesgo, de modo que el crecimiento no compromete la solución. La regla hebbiana no es entonces una regla de corrección de error y su utilidad depende por completo de la distribución de clases del problema.

5.2 Comportamiento ante cambios del umbral y de la tasa de aprendizaje

El umbral tuvo un efecto menor en los dos modelos, pero por razones distintas. En el Perceptrón el umbral y el sesgo son intercambiables, de modo que las primeras correcciones absorben cualquier valor del umbral. En el Adaline el umbral no participa en el ajuste y solo interviene al convertir la salida lineal en una clase, papel tardío que permite recuperar patrones sin volver a entrenar.

La tasa de aprendizaje, en cambio, desempeña papeles opuestos en los dos modelos. En el Perceptrón un valor alto solo acelera la convergencia y en ningún caso produjo inestabilidad, porque la corrección se aplica únicamente cuando hay error y siempre apunta en la dirección que lo reduce, de manera que un paso grande a lo sumo sobrepasa la frontera y se corrige en el patrón siguiente. En el Adaline existe un límite de estabilidad propio del descenso de gradiente, que depende de la magnitud de las entradas. Por debajo de ese límite el error desciende hacia el mínimo, cerca de él el descenso oscila y el error final se estabiliza por encima del óptimo, como ocurre con la tasa 0.5, y por encima del límite cada paso sobrepasa el mínimo y cae más lejos que el anterior, de modo que el error crece época tras época, que es lo que ocurre con la tasa uno en la Tabla 8.

5.3 Diferencias en la forma de la curva de error

La Figura 4 resume la diferencia esencial entre ambos modelos. El Adaline presenta una curva continua, suave y monótona, con un piso positivo que corresponde al mínimo alcanzable por una función lineal sobre una tabla de verdad. El Perceptrón no tiene una curva de error propiamente dicha, sino un conteo entero de patrones mal clasificados que sube y baja sin garantía de mejora época a época. Esa falta de monotonía no es un defecto de implementación, sino la consecuencia de corregir sobre una función no diferenciable, en la cual una corrección que arregla un patrón puede dañar otro que ya estaba bien clasificado.

De ahí se desprende una observación sobre la calidad de las soluciones. El Perceptrón entrega el primer hiperplano que separa los datos y el Adaline entrega el que minimiza la distancia cuadrática a los valores deseados, criterio que no coincide con separar las clases. Esa discrepancia explica los tres casos en los que el Adaline deja un patrón mal clasificado en compuertas que el Perceptrón resuelve sin error, y se agudiza al aumentar el número de entradas porque la tabla de verdad tiene más vértices.

5.4 Efecto del punto de partida de los pesos

Este fue el resultado más marcado del trabajo y separa claramente a los dos modelos. En el Perceptrón, partir de pesos muy cercanos a cero es la mejor condición inicial de las probadas. Con pesos nulos la suma ponderada vale cero para todos los patrones, queda por debajo del umbral y la neurona empieza apagada, de modo que solo recibe correcciones de los patrones cuya salida deseada es uno y construye los pesos a partir de la evidencia de los datos. El resultado es una convergencia en cinco épocas para AND y doce para OR, con el mismo hiperplano final en todas las semillas. Con pesos grandes frente a la tasa de aprendizaje ocurre lo contrario, pues hacen falta muchas correcciones pequeñas para vencer el punto de partida, el modelo tarda más de cuarenta épocas, algunas semillas no alcanzan a converger dentro del tope y cada semilla termina en un separador distinto. Conviene matizar que el caso de pesos exactamente nulos elimina la aleatoriedad por construcción, así que el hallazgo verdaderamente informativo es el de escala 0.001, donde las semillas siguen siendo distintas y aun así convergen al mismo sitio y en el mismo número de épocas.

El Adaline se comporta de manera opuesta y llega a los mismos pesos, con diferencias de una milésima, desde cualquiera de las cuatro escalas iniciales, incluida la más grande. La razón es que la función de costo de la ecuación (6) es cuadrática en los pesos y tiene un único mínimo, de modo que el punto de partida solo determina la longitud del camino y no el destino. Ese contraste ilustra bien la diferencia de fondo entre optimizar una función continua y corregir errores de clasificación, y también matiza la afirmación de la plantilla según la cual la inicialización no es crítica en un modelo de una sola neurona. Para el Adaline la afirmación es exacta. Para el Perceptrón la inicialización no compromete la convergencia, que el teorema garantiza, pero sí afecta de manera apreciable la velocidad, la reproducibilidad y la solución concreta que se obtiene.

5.5 Comportamiento sobre datos reales

La exactitud de prueba varió entre 94.4 y 98.9 por ciento de una partición a otra sin seguir una tendencia clara, y las diferencias entre entrenamiento y prueba se mantuvieron dentro de 2.5 puntos porcentuales. No hay indicios de sobreajuste, algo esperable en un modelo de cinco parámetros que debe describir hasta 1235 patrones de entrenamiento, pues su capacidad es insuficiente para memorizar los datos. Sí existe, en cambio, un subajuste de origen estructural, ya que las clases no son linealmente separables y ningún hiperplano puede clasificarlas por completo, lo cual se manifiesta en el hecho de que el Perceptrón nunca converge y deja entre 0.7 y 5.7 por ciento de error tanto en entrenamiento como en prueba.

La partición 90-10 merece una advertencia, pues su conjunto de prueba tiene solo 137 billetes y cada error individual pesa 0.73 puntos porcentuales, de modo que sus cifras son las menos estables. Ese es el compromiso que la comparación entre proporciones permite visualizar. El contraste en el efecto del escalado, por último, es coherente con la teoría y quedó medido en los dos modelos. El Perceptrón decide por el signo de la suma ponderada, de modo que un cambio de escala en las entradas se compensa con un cambio de escala en los pesos y la frontera resultante es la misma. Por eso sus cifras sin escalar no solo no empeoran sino que mejoran un punto, ya que con entradas grandes cada corrección avanza más y cien épocas rinden más trabajo efectivo. En el Adaline la corrección es proporcional al valor de la entrada y entra directamente en el paso de gradiente, así que un atributo con valores hasta 17.93 amplifica ese paso y empuja al algoritmo por encima de su límite de estabilidad. El escalado, en consecuencia, no es un trámite de preparación sino una condición necesaria para que la Regla Delta opere con una tasa razonable.

6. Conclusiones

Las reglas 2 y 3 resolvieron las seis compuertas AND y OR de 2, 3 y 4 entradas en menos de treinta y una épocas promedio y con la totalidad de las semillas, mientras que la regla hebbiana solo resolvió la compuerta OR. La causa no es la implementación sino la ausencia del término de error, que hace crecer los pesos sin límite cuando la clase positiva está poco representada, hasta valores cercanos a cien en la compuerta AND.

El Adaline clasificó correctamente AND de 2 y 3 entradas y OR de 2 entradas, y dejó un patrón mal clasificado en AND de 4 y en OR de 3 y 4. Minimizar el error cuadrático de la salida lineal no equivale a minimizar el error de clasificación, y por eso el modelo puede fallar en problemas que el Perceptrón resuelve sin error. Mover el umbral de decisión sobre el modelo ya entrenado recuperó esos patrones en dos de los tres casos.

La tasa de aprendizaje cumple funciones opuestas en los dos modelos. En el Perceptrón un valor mayor solo reduce el número de épocas y nunca produjo inestabilidad. En el Adaline existe un límite de estabilidad, de modo que con 0.5 el error final quedó por encima del mínimo y con uno el error creció sin control.

Empezar con pesos muy cercanos a cero fue la mejor condición inicial para el Perceptrón en estos problemas. Convergió en cinco épocas en AND y doce en OR, con el mismo hiperplano en todas las semillas, frente a más de cuarenta épocas, un diez por ciento de ejecuciones sin converger y un separador distinto por semilla cuando los pesos iniciales fueron grandes respecto de la tasa de aprendizaje.

El Adaline resultó insensible al punto de partida y terminó en los mismos pesos, con diferencias de una milésima, desde escalas iniciales que van de cero a cinco. Ese resultado confirma de forma empírica que el descenso de gradiente sobre un error cuadrático llega siempre al mismo mínimo, y establece la diferencia práctica más importante entre los dos modelos en cuanto a reproducibilidad de los experimentos.

Sobre el conjunto de billetes los dos modelos alcanzaron entre 94.4 y 98.9 por ciento de exactitud de prueba, sin sobreajuste y sin una tendencia clara respecto de la proporción de partición. La matriz de confusión reveló perfiles de error distintos, con el Perceptrón repartiendo sus seis errores entre ambas clases y el Adaline concentrando los diecisiete suyos en una sola, sin dejar pasar ningún billete falso.

Sobre los conjuntos de Iris que genera el script de particiones de clase, el Perceptrón resolvió el problema de *setosa* contra el resto en dos épocas y con cero errores en las cuatro proporciones, lo que confirma el teorema de convergencia sobre datos reales separables y delimita por contraste el caso de los billetes.

El escalado de las entradas no perjudicó al Perceptrón, que incluso mejoró un punto sin escalar, y resultó imprescindible para el Adaline, que sin escalar divergió con su tasa base en las cuatro particiones. El control negativo con la compuerta XOR confirmó, por último, que ninguno de los dos modelos supera el límite de la separabilidad lineal.

7. Referencias

- Hebb, D. O. (1949). *The Organization of Behavior*. Wiley.
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408.

- Novikoff, A. B. (1962). On convergence proofs for perceptrons. *Proceedings of the Symposium on the Mathematical Theory of Automata*, 12, 615–622.
- Widrow, B., & Hoff, M. E. (1960). Adaptive switching circuits. *IRE WESCON Convention Record*, 4, 96–104.
- Widrow, B., & Lehr, M. A. (1990). 30 years of adaptive neural networks: Perceptron, Madaline, and Backpropagation. *Proceedings of the IEEE*, 78(9), 1415–1442.
- Minsky, M., & Papert, S. (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Haykin, S. (2009). *Neural Networks and Learning Machines* (3.^a ed.). Pearson.
- Lohweg, V. (2013). Banknote Authentication. UCI Machine Learning Repository.

A. Tablas complementarias

Tabla 15: Perceptrón con la regla 3 sobre el conjunto de billetes. Exactitud de prueba en porcentaje según la tasa de aprendizaje

Entradas	Partición	Tasa de aprendizaje α		
		0.01	0.1	1
Sin escalar	60-40	97.3	98.4	98.2
	70-30	98.5	98.5	98.5
	80-20	98.9	98.5	98.9
	90-10	99.3	98.5	97.1
Escaladas	60-40	96.9	94.4	97.3
	70-30	97.1	98.5	98.8
	80-20	97.1	98.9	98.2
	90-10	96.4	97.8	95.6

Tabla 16: Perceptrón con la regla 3 y tasa 0.1. Épocas promedio de diez semillas según el umbral de la función escalón

Compuerta	Umbral θ		
	0	0.5	1
AND 2	15.4	13.7	14.7
AND 3	16.6	17.7	17.1
AND 4	12.0	15.6	13.2
OR 2	12.6	13.0	12.5
OR 3	19.9	16.8	19.5
OR 4	15.8	30.4	15.0

B. Código de los bloques completados

Se reproducen completos los cuatro bloques que la plantilla dejaba pendientes, tal como quedaron en el *live script* entregado. Ningún nombre ni argumento de función fue modificado.

Tabla 17: Adaline sobre el conjunto de billetes. Exactitud de prueba en porcentaje según la tasa de aprendizaje. La marca diverge indica que el error creció sin control durante el entrenamiento

Entradas	Partición	Tasa de aprendizaje α			
		0.0001	0.001	0.01	0.05
Sin escalar	60-40	96.5	97.5	96.4	diverge
	70-30	96.1	95.9	72.3	diverge
	80-20	96.7	96.7	96.4	diverge
	90-10	94.9	95.6	91.2	diverge
Escaladas	60-40	81.1	95.6	97.3	96.5
	70-30	82.3	96.6	95.9	95.9
	80-20	83.2	96.7	96.7	96.7
	90-10	88.3	95.6	94.9	94.2

Listing 1: Inicialización de los pesos y el sesgo del Perceptrón

```

1 function [W, b] = init_perceptron(cfg)
2     % Inicializa pesos y sesgo con valores aleatorios pequeños.
3     W = randn(1, cfg.n_inputs) * cfg.init_scale;
4     b = randn(1) * cfg.init_scale;
5 end

```

Listing 2: Las tres reglas de ajuste de pesos del Perceptrón

```

1 function delta = compute_delta(cfg, d, y, x)
2     % Calcula el incremento de pesos Delta_W según la regla seleccionada
3     % en cfg.learning_rule. `x` es un patrón (vector fila), `d` la
4     % salida deseada y `y` la salida actual del Perceptrón (ya
5     % escalonada).
6     switch cfg.learning_rule
7         case 'hebb'
8             delta = d * x;
9         case 'perceptron'
10            delta = (d - y) * x;
11         case 'perceptron_alpha'
12            delta = cfg.alpha * (d - y) * x;
13         otherwise
14            error('Regla de aprendizaje no reconocida: %s', cfg.learning_rule);
15     end
16 end

```

Listing 3: Entrenamiento del Adaline con la Regla Delta sobre la salida lineal

```

1 function [W, b, mse_hist] = train_adaline(X, D, cfgA)
2     [W, b] = init_adaline(cfgA);
3     mse_hist = zeros(cfgA.max_epochs, 1);
4
5     for epoch = 1:cfgA.max_epochs
6         epoch_error = 0;
7         for p = 1:size(X, 1)
8             x = X(p, :);
9             d = D(p);
10
11             net = W * x' + b;    % salida LINEAL (sin escalón)
12             y = net;
13
14             delta = cfgA.alpha * (d - y) * x;
15             W = W + delta;
16             b = b + cfgA.alpha * (d - y);
17
18             epoch_error = epoch_error + 0.5 * (d - y)^2;
19         end
20
21         mse_hist(epoch) = epoch_error / size(X, 1);
22         if mse_hist(epoch) <= cfgA.target_mse
23             mse_hist = mse_hist(1:epoch);
24             break;
25         end
26     end
27 end

```

Listing 4: Partición aleatoria en entrenamiento y prueba

```

1 function [Xtr, Dtr, Xte, Dte] = split_dataset(X, D, train_ratio, seed)
2     % Divide (X, D) en un subconjunto de entrenamiento y uno de prueba,
3     % respetando la proporción train_ratio (p. ej. 0.7 = 70% entrenamiento).
4     rng(seed);
5     n = size(X, 1);
6     idx = randperm(n);
7     n_train = round(train_ratio * n);
8     idx_train = idx(1:n_train);
9     idx_test = idx(n_train+1:end);
10    Xtr = X(idx_train, :); Dtr = D(idx_train, :);
11    Xte = X(idx_test, :); Dte = D(idx_test, :);
12 end

```